



DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS

College of Arts and Sciences | University of the Philippines Visayas

Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020

Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

Evaluator

CMSC 124 Lab 3

Prepared by: Rene Andre Bedonia Jocsing

Published: October 07, 2025

Deadline: November 28, 2025

Welcome to the third chapter of your language hacking journey. Your scanner can tokenize source code, your parser can build syntax trees, and now it's time for the grand finale: **evaluation**. This is where your language comes to life—where those abstract syntax trees finally compute actual values.

Think of evaluation as the moment Dr. Frankenstein throws the switch. Your previously lifeless syntax tree will open its eyes, take a breath, and execute code. No lightning required (though it adds to the ambiance).

From Trees to Values

So far, you've built a pipeline that transforms source code into abstract syntax trees. But trees alone don't *do* anything. They're just data structures sitting in memory. To make your language useful, you need to traverse these trees and compute the values they represent.

Consider this simple expression: $2 + 3 * 4$

Your parser built a tree that correctly captures precedence:

```
1      +
2     / \
3    2  *
4       / \
5      3  4
```

Now your evaluator needs to walk this tree and compute the result: 14. The process is straightforward—evaluate the leaves first (the literals 2, 3, 4), then work your way up, applying each operator to its operands.

This is called a **post-order traversal** because each node evaluates its children before doing its own work. It's the natural way of evaluating mathematical expressions, and it generalizes perfectly to more complex programming language constructs.

Representing Values

Before you can compute values, you need to decide how to represent them in Kotlin. This is where dynamic typing gets interesting.

In your programming language, values are dynamically typed—a variable can hold a number now and a string later. But Kotlin is instead statically typed. How do you bridge this gap?

If you're using a language like Java or C#, you can use their `Object` type as your universal value representation. If you're using Python or JavaScript, the implementing language is already dynamically

typed, so this problem doesn't exist. If you're using C++ or Rust, you might implement a variant type or a union. But in Kotlin, you can easily use the `Any?` type.

To summarize, you need:

- A way to store values of different types in the same variable
- A way to determine a value's type at runtime
- Conversion between your language's representation and your host language's types

Evaluator Architecture

Now you need to implement the logic that traverses your AST and computes values. There are several architectural approaches you can take, each with different trade-offs.

If you used the **Visitor pattern** in your parser, you can extend it here. Create an evaluator class that implements visitor methods for each AST node type. This keeps evaluation logic separate from your AST classes.

If you used the **Interpreter pattern**, you likely have an `evaluate()` or `interpret()` method directly on each AST node class. You can extend these methods to return computed values instead of just printing representations.

You might also implement evaluation using a **switch statement** or **method dispatch** based on node types, or use **function pointers/lambdas**.

Regardless of your architectural choice, the core principle remains the same: each AST node type needs corresponding logic that knows how to evaluate that construct. Your evaluation code structure should mirror your grammar—each production rule gets corresponding evaluation logic.

Your chosen approach should provide:

- Clear separation between parsing and evaluation concerns
- Easy maintainability as your language grows
- Consistent handling of the recursive tree traversal
- Clean error propagation when evaluation fails

Evaluating Different Expression Types

Let's walk through how to evaluate each kind of expression your parser can produce.

Literals are the easiest. A literal expression contains a value that was determined during scanning. To evaluate it, just return that value. No computation needed.

Grouping expressions (parentheses) are almost as simple. Just recursively evaluate the expression inside the parentheses and return the result. The parentheses themselves don't affect the value—they only influenced parsing precedence.

Unary expressions like `-5` or `!true` require you to:

1. Recursively evaluate the operand
2. Apply the unary operator to the result
3. Return the new value

For negation, make sure the operand is a number. For logical not, you'll need to decide what counts as "truthy" in your language.

Binary expressions are similar but with two operands:

1. Recursively evaluate the left operand
2. Recursively evaluate the right operand
3. Apply the binary operator
4. Return the result

Arithmetic operators (`+`, `-`, `*`, `/`) work on numbers. The `+` operator might also concatenate strings, depending on your language design. Comparison operators (`<`, `<=`, `>`, `>=`) compare numbers and return

booleans. Equality operators (`==`, `!=`) work on any types.

Truthiness

Most dynamically typed languages partition all values into “truthy” and “falsey” categories. This determines how values behave in boolean contexts like `if` statements or the `!` operator.

The exact rules vary by language, but a common approach is:

- `false` and `null` are falsey
- Everything else is truthy

Some languages also treat empty strings, zero, or empty collections as falsey, but simplicity has its virtue.

Runtime Errors

Not all expressions can be successfully evaluated. What happens when the user tries to negate a string or add a number to a boolean? These are **runtime errors**—problems that can only be detected during execution.

When a runtime error occurs, you need to:

1. Stop evaluating the current expression
2. Report a meaningful error message to the user
3. Avoid crashing the entire interpreter

The third point is crucial. If users are running a REPL, they should be able to fix their mistake and continue. Don't let one bad expression kill the whole session.

Your error handling strategy will depend on your implementation. Exceptions work well. Error values or result types work too. The key is to unwind the evaluation stack cleanly when an error occurs.

Laboratory Deliverables

Expression Evaluator

Building on your parser from the previous lab, implement an evaluator that can compute the values of expressions in your language. Your evaluator should handle:

- Literal values (numbers, strings, booleans, nil)
- Unary expressions (negation, logical not)
- Binary expressions (arithmetic, comparison, equality)
- Grouping (parenthesized expressions)
- Proper operator precedence and associativity
- Other constructs as needed in your language

Runtime Error Handling

Implement graceful runtime error handling for type mismatches and other evaluation errors. Your interpreter should:

- Detect when operands have incorrect types for operators
- Report clear, helpful error messages that include line numbers
- Continue running after encountering errors (don't crash)
- Handle edge cases like division by zero (your choice how)

Integration and Testing

Update your REPL to evaluate expressions and print their results. Instead of showing the AST structure, users should now see computed values. Your evaluator marks a major milestone—you now have a working (if limited) programming language!

Expected Output

```
1  > 42
2  42
3  > 3.14159
4  3.14159
5  > "Hello, world!"
6  Hello, world!
7  > true
8  true
9  > false
10 false
11 > nil
12 nil
13 > -123
14 -123
15 > !true
16 false
17 > !false
18 true
19 > !nil
20 true
21 > !"hello"
22 true
23 > 1 + 2
24 3
25 > 3 - 1
26 2
27 > 2 * 3
28 6
29 > 8 / 4
30 2
31 > "Hello" + " " + "world"
32 Hello world
33 > 1 + 2 * 3
34 7
35 > (1 + 2) * 3
36 9
37 > 1 < 2
38 true
39 > 2 > 3
40 false
41 > 1 <= 1
42 true
43 > 2 >= 1
44 true
45 > 1 == 1
46 true
47 > 1 != 2
48 true
49 > "hello" == "hello"
50 true
51 > "hello" != "world"
52 true
53 > nil == nil
54 true
55 > true == false
56 false
57 > 1 == "1"
58 false
59 > (5 > 3) == true
```

```

60 true
61 > 3 + 4 * 5 == 23
62 true
63 > -1 + 2
64 1
65 > -(3 + 4)
66 -7
67 > 1 / 0
68 [line 1] Runtime error: Division by zero.
69 > "hello" + 5
70 [line 1] Runtime error: Operands must be two numbers or two strings.
71 > 5 > "hello"
72 [line 1] Runtime error: Operands must be numbers.
73 > -"hello"
74 [line 1] Runtime error: Operand must be a number.

```

Rubric for Programming Exercises (50 pts)

Criteria (10 Points Each)	Excellent (9-10)	Good (6-8)	Fair (3-5)	Poor (0-2)
Program Correctness	Program executes correctly with no syntax or runtime errors, meets/exceeds specifications, and displays correct output	Program executes and outputs with minor errors, yet meets specifications	Program executes and outputs with major errors, yet somehow meets specifications	Program does not execute or does not meet specs
Logical Design	Program is logically well-designed with excellent structure and flow	Program has slight logic errors that do not significantly affect the results	Program has significant logic errors affecting functionality	Program logic is fundamentally incorrect
Code Mastery	Programmer demonstrates excellent mastery over the program's code	Programmer demonstrates adequate mastery over the program's code	Programmer demonstrates fair mastery over the program's code	Programmer demonstrates poor mastery over the program's code
Engineering Standards	Program is stylistically well designed from an engineering standpoint	Slight inappropriate design choices (i.e., poor variable names, improper indentation)	Severe inappropriate design choices (i.e., code repetition, redundancy)	Program is poorly written
Documentation*	Program is well-documented: comments exist for clarity, not redundancy	Missing one required comment or some redundant comments	Missing two or more required comments or many redundant comments	Most documentation missing or most documentation is redundant

***Remember:** "Code tells you how, comments tell you why." — Jeff Atwood, co-founder of Stack

Overflow and Discourse